

МИНОБРНАУКИ РОССИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«Пермский государственный национальный
исследовательский университет»

Институт компьютерных наук и
технологий

ОТЧЁТ
по индивидуальной работе №2
по дисциплине «Языки программирования»
Вариант 15, 19

Работу выполнил
студент группы ИТ-7,8-2025 1 курса
Фотин М.С.
«21» июня 2026 г.

Работу проверила
Шилина А.В.
«__» _____ 2026 г.

Пермь 2026

СОДЕРЖАНИЕ

Часть 1. 15 вариант. Калькулятор постфиксной записи	3
Постановка задачи	3
Алгоритм решения.....	3
Тестирование.....	4
Код программы	8
Часть 2. Вариант 19. Бинарное дерево.....	9
Постановка задачи	9
Алгоритм решения.....	9
Тестирование.....	10
Код программы	14

Часть 1. 15 вариант. Калькулятор постфиксной записи

Постановка задачи

Используя структуру стека подсчитать значение арифметического выражения, записанного в обратной польской записи (постфиксной записи) при условии, что используются знаки операций $+$, $-$, $*$, $/$ и операнды являются вещественными положительными числами. Во вводимой пользователем строке числа и знаки операций разделяются одним пробелом.

Алгоритм решения

В основе решения лежит собственная динамическая структура данных - стек, реализованный на базе односвязного списка (классы `StackNode` и `CustomStack`). Алгоритм обработки выражения состоит из следующих шагов:

1. Входная строка разбивается на токены (отдельные числа и операторы) с использованием пробела в качестве разделителя.
2. Происходит последовательный перебор полученных токенов. Если токен является числом, оно проверяется на то, что является вещественным и положительным, после чего помещается в стек (операция `push`).
3. Если токен является знаком арифметической операции ($+$, $-$, $*$, $/$), из стека извлекаются два верхних элемента (операция `pop`). Первым извлекается правый операнд, вторым - левый.
4. Выполняется соответствующая операция, и её результат заносится обратно в стек.
5. После завершения цикла обработки в стеке должен остаться ровно один элемент, который и является результатом вычисления всего выражения. Оставшиеся элементы свидетельствуют о синтаксической ошибке в исходном выражении.

Для защиты от некорректного пользовательского ввода реализована обработка исключений (ввод неположительных чисел, деление на ноль, нехватка операндов, лишние операнды).

Тестирование

Тестирование проводилось с учетом комплексной проверки функционала, включая проверку базовых сценариев и обработки исключительных ситуаций

Тест 1. Корректное выражение (простейшее).

- Ввод: 5 2 +
- Ожидаемый результат: 7.0
- Фактический результат: 7.0

```
МЕНЮ: ВАРИАНТ 15 (ОБРАТНАЯ ПОЛЬСКАЯ ЗАПИСЬ)
=====
1. Вычислить выражение
2. Выход
=====
Выберите действие (1-2): 1
Введите выражение. Числа и знаки разделяйте одним пробелом.
Пример: 5 2 / (означает 5 / 2)
Ввод: 5 2 +
Результат: 7.0
```

Тест 2. Выражение с несколькими операторами.

- Ввод: 5 2 3 + * (то же самое, что $5 * (2 + 3)$)
- Ожидаемый результат: 25.0
- Фактический результат: 25.0

```
=====
МЕНЮ: ВАРИАНТ 15 (ОБРАТНАЯ ПОЛЬСКАЯ ЗАПИСЬ)
=====
1. Вычислить выражение
2. Выход
=====
Выберите действие (1-2): 1
Введите выражение. Числа и знаки разделяйте одним пробелом.
Пример: 5 2 / (означает 5 / 2)
Ввод: 5 2 3 + *
Результат: 25.0
```

Тест 3. Деление на ноль.

- Ввод: 3 0 /.
- Ожидаемый результат: Деление на ноль невозможно
- Результат: Деление на ноль невозможно

```
=====
МЕНЮ: ВАРИАНТ 15 (ОБРАТНАЯ ПОЛЬСКАЯ ЗАПИСЬ)
=====
1. Вычислить выражение
2. Выход
=====
Выберите действие (1-2): 1
Введите выражение. Числа и знаки разделяйте одним пробелом.
Пример: 5 2 / (означает 5 / 2)
Ввод: 3 0 /
Ошибка выполнения: Деление на ноль невозможно
```

Тест 4. Отрицательное число.

- Ввод: -5 2 +.
- Ожидаемый результат: Некорректный ввод: “операнд” не является положительным числом
- Результат: Некорректный ввод: “-5” не является положительным числом

```
=====
МЕНЮ: ВАРИАНТ 15 (ОБРАТНАЯ ПОЛЬСКАЯ ЗАПИСЬ)
=====
1. Вычислить выражение
2. Выход
=====
Выберите действие (1-2): 1
Введите выражение. Числа и знаки разделяйте одним пробелом.
Пример: 5 2 / (означает 5 / 2)
Ввод: -5 2 +
Ошибка выполнения: Некорректный ввод: -5. '-5' не является положительным числом
=====
```

Тест 5. Недостаточно операндов.

- Ввод: 1 2 + +.
- Ожидаемый результат: Стек пуст, недостаточно операндов
- Результат: Стек пуст, недостаточно операндов

```
МЕНЮ: ВАРИАНТ 15 (ОБРАТНАЯ ПОЛЬСКАЯ ЗАПИСЬ)
=====
1. Вычислить выражение
2. Выход
=====
Выберите действие (1-2): 1
Введите выражение. Числа и знаки разделяйте одним пробелом.
Пример: 5 2 / (означает 5 / 2)
Ввод: 1 2 + +
Ошибка выполнения: Стек пуст, недостаточно операндов
```

Тест 6. Лишние операнды.

- Ввод: 1 2 3.
- Ожидаемый результат: Слишком много операндов и не хватает знаков операций
- Результат: Слишком много операндов и не хватает знаков операций

```
=====
МЕНЮ: ВАРИАНТ 15 (ОБРАТНАЯ ПОЛЬСКАЯ ЗАПИСЬ)
=====
1. Вычислить выражение
2. Выход
=====
Выберите действие (1-2): 1
Введите выражение. Числа и знаки разделяйте одним пробелом.
Пример: 5 2 / (означает 5 / 2)
Ввод: 1 2 3
Ошибка выполнения: Слишком много операндов и не хватает знаков операций
```

Тест 7. Недопустимый символ.

- Ввод: 5 2 \$ +.
- Ожидаемый результат: Недопустимый ввод: '\$'.
- Результат: Недопустимый ввод: '\$'.

```
=====
МЕНЮ: ВАРИАНТ 15 (ОБРАТНАЯ ПОЛЬСКАЯ ЗАПИСЬ)
=====
1. Вычислить выражение
2. Выход
=====
Выберите действие (1-2): 1
Введите выражение. Числа и знаки разделяйте одним пробелом.
Пример: 5 2 / (означает 5 / 2)
Ввод: 3 $ 2
Ошибка выполнения: Некорректный ввод: $
=====
```

Тест 8. Пустая строка.

- Ввод: “пустая строка”.
- Ожидаемый результат: Пустая строка. Вы ничего не ввели.
- Результат: Пустая строка. Вы ничего не ввели.

```
МЕНЮ: ВАРИАНТ 15 (ОБРАТНАЯ ПОЛЬСКАЯ ЗАПИСЬ)
=====
1. Вычислить выражение
2. Выход
=====
Выберите действие (1-2): 1
Введите выражение. Числа и знаки разделяйте одним пробелом.
Пример: 5 2 / (означает 5 / 2)
Ввод:
Ошибка выполнения: Пустая строка. Вы ничего не ввели.
=====
```

Тест 9. Неверный код меню

- Ввод: 7
- Ожидаемый результат: Неверный пункт меню.
- Результат: Неверный пункт меню.

```
=====
МЕНЮ: ВАРИАНТ 15 (ОБРАТНАЯ ПОЛЬСКАЯ ЗАПИСЬ)
=====
1. Вычислить выражение
2. Выход
=====
Выберите действие (1-2): 7
-> Ошибка: Неверный пункт меню.
=====
```

Код программы

Код Github: https://github.com/1matuxxa/ikm_python

Часть 2. Вариант 19. Бинарное дерево

Постановка задачи

В файле даны N целых чисел, и здесь же указан путь их размещения в бинарном дереве виде двоичного кода (коды не повторяются). Необходимо построить двоичное дерево целых чисел, в котором путь по дереву определяется указанным двоичным кодом в этом листе (1 — переход к правому потомку, 0 — переход к левому потомку). В корень автоматически заносится значение 0. Следует учитывать ситуацию, когда дерево не может быть построено.

Алгоритм решения

В основе решения лежит собственная реализация бинарного дерева. Алгоритм построения дерева следующий:

1. Создается объект дерева, в корневой узел которого автоматически записывается значение 0.
2. Для каждой поступающей пары данных (значение и строка двоичного пути) алгоритм начинает обход от корня.
3. Строка пути перебирается посимвольно. Встретив '0', алгоритм пытается перейти к левому потомку. Встретив '1', — к правому.
4. Если нужного потомка не существует, алгоритм динамически создает пустой узел (без значения).
5. После прохождения всего пути алгоритм проверяет целевой узел. Если узел пуст, в него записывается переданное значение. Если в узле уже есть значение, фиксируется конфликт путей (ошибка построения дерева).
6. Для подтверждения правильности структуры дерева реализован метод симметричного обхода (In-Order), возвращающий список значений узлов в порядке "левый-корень-правый".

Тестирование

Тест 1. Базовое построение дерева

- Ввод: Узел 1: Значение 5, Путь: 0
Узел 2: Значение 10, Путь: 1
- Ожидаемый результат: Узлы успешно добавлены.
Обход In-Order: [5, 0, 10]
- Результат: Симметричный обход: [5, 0, 10]

```
=====
Выберите действие (1-2): 1
Введите количество чисел для добавления: 2
--- Элемент 1 ---
Значение (целое число): 5
Двоичный код пути (0 и 1): 0
-> Добавлено.
--- Элемент 2 ---
Значение (целое число): 10
Двоичный код пути (0 и 1): 1
-> Добавлено.
=== ИТОГ ===
Дерево построено
Симметричный обход: [5, 0, 10]
```

Тест 2. Конфликт путей

• Ввод: Узел 1: Значение 5, Путь: 10

Узел 2: Значение 8, Путь: 10

• Ожидаемый результат: Ошибка: Путь '10' уже занят. Значение не добавлено

Попробуйте начать заново или продолжите с текущим деревом

• Результат: Ошибка: Путь '10' уже занят. Значение не добавлено

Попробуйте начать заново или продолжите с текущим деревом

```
=====
1. Построить дерево и вывести обход
2. Выход
=====
Выберите действие (1-2): 1
Введите количество чисел для добавления: 2
--- Элемент 1 ---
Значение (целое число): 5
Двоичный код пути (0 и 1): 10
-> Добавлено.
--- Элемент 2 ---
Значение (целое число): 8
Двоичный код пути (0 и 1): 10
-> Ошибка при добавлении: Путь '10' уже занят значением 5. Дерево не может быть построено
-> Попробуйте начать заново или продолжить с текущим деревом
=== ИТОГ ===
Дерево построено
Симметричный обход: [0, 5]
```

Тест 3. Недопустимые символы в пути

• Ввод: Значение: 12, Путь: 021

• Ожидаемый результат: Ошибка: Недопустимый символ '2'.

• Результат: Ошибка: Недопустимый символ '2'. Разрешены только «0» и «1»

```
=====
МЕНЮ: ВАРИАНТ 19 (БИНАРНОЕ ДЕРЕВО КОДОВ)
=====
1. Построить дерево и вывести обход
2. Выход
=====
Выберите действие (1-2): 1
Введите количество чисел для добавления: 2
--- Элемент 1 ---
Значение (целое число): 12
Двоичный код пути (0 и 1): 021
-> Ошибка при добавлении: Недопустимый символ '2'. Разрешены только '0' и '1'
-> Попробуйте начать заново или продолжить с текущим деревом
--- Элемент 2 ---
Значение (целое число): █
```

Тест 4: Ввод недопустимого значения при выборе количества чисел

- Ввод: «gg»
- Ожидаемый результат: Ошибка! Введите целое положительное число
- Результат: Ошибка! Введите целое положительное число

```
=====
МЕНЮ: ВАРИАНТ 19 (БИНАРНОЕ ДЕРЕВО КОДОВ)
=====
1. Построить дерево и вывести обход
2. Выход
=====
Выберите действие (1-2): 1
Введите количество чисел для добавления: gg
-> Ошибка: Количество должно быть целым положительным числом
```

Тест 5: Ввод недопустимого значения при добавлении числа

- Ввод: «gg»
- Ожидаемый результат: Ошибка! Введите целое положительное число
- Результат: Ошибка! Введите целое положительное число

```
1. Построить дерево и вывести обход
2. Выход
=====
Выберите действие (1-2): 1
Введите количество чисел для добавления: 3
--- Элемент 1 ---
Значение (целое число): gg
Двоичный код пути (0 и 1): 1
-> Ошибка при добавлении: Введите допустимое число!
-> Попробуйте начать заново или продолжить с текущим деревом
```

Тест 6: Пустая строка

- Ввод: «»
- Ожидаемый результат: Пустая строка. Вы ничего не ввели
- Результат: Пустая строка. Вы ничего не ввели

```
=====
МЕНЮ: ВАРИАНТ 19 (БИНАРНОЕ ДЕРЕВО КОДОВ)
=====
1. Построить дерево и вывести обход
2. Выход
=====
Выберите действие (1-2): 1
Введите количество чисел для добавления:
-> Ошибка: Пустая строка. Вы ничего не ввели.
=====
МЕНЮ: ВАРИАНТ 19 (БИНАРНОЕ ДЕРЕВО КОДОВ)
=====
1. Построить дерево и вывести обход
2. Выход
=====
Выберите действие (1-2): █
```

Тест 7: Недопустимый ввод в меню

- Ввод: «3»
- Ожидаемый результат: Неверный пункт меню
- Результат: Неверный пункт меню

```
=====
МЕНЮ: ВАРИАНТ 19 (БИНАРНОЕ ДЕРЕВО КОДОВ)
=====
1. Построить дерево и вывести обход
2. Выход
=====
Выберите действие (1-2): 3
-> Ошибка: Неверный пункт меню
=====
МЕНЮ: ВАРИАНТ 19 (БИНАРНОЕ ДЕРЕВО КОДОВ)
=====
1. Построить дерево и вывести обход
2. Выход
=====
Выберите действие (1-2): █
```

Код программы

Github: https://github.com/1matuxxa/ikm_python